

HW5 Butterworth Filter Design Flow

1.

Here we intend to design a discrete-time low-pass filter according to a continuous-time Butterworth filter. Once the cut-off frequency is set as 0.2π , a helpful transformation named bilinear transformation maps this frequency to discrete-time domain. The relationship of frequency between in discrete-time domain and in continuous-time domain can be described as

$$\Omega = \frac{2}{T_d} \tan(\omega/2) \quad (1)$$

where Ω and ω represent frequency in discrete-time domain and continuous-time domain respectively, and T_d is a sampling interval which samples the impulse response (or filter here).

Once again, the goal is to devise a discrete-time low-pass Butterworth filter with cut-off frequency at 0.2π , so ω is 0.2π undoubtedly. However, the given notation Ω_c is a continuous-time frequency. Wherever the cut-off frequency is in s domain or z domain it should be 0.2π as much as possible, so that

$$T_d = \frac{2}{\Omega_c} \tan(\Omega_c/2)$$

T_d is about 1.0343. Fortunately, once we decide the discrete-time cut-off frequency, the term ΩT_d is a constant in equation (1). In the other word, any given Ω would obtain a corresponding T_d due to the nonlinear transformation, a finite domain being the range of an infinite domain. As a consequence, in my opinion, that is why the built-in function *butter* only requires the normalized cut-off frequency. **We will compare the result obtained from our design with that from built-in function in question 5 to verify our design.** For the sake of stability, all poles are at the left half of the s-plane and uniformly distributed in angle on a circle of radius Ω_c . Then bilinear transformation is implemented to map these poles to the left half of z-plane. A powerful function *poly* is used to build two polynomials referring to these poles or zeros, i.e. numerator and denominator of filter. Finally, this function, FunctionForQ1, returns DEN and NUM for any given N and cut-off frequency.

```
function [Td, DEN, NUM] = FunctionForQ1(N, omega, Omega)
% N : filter order
% omega : frequency in discrete-time domain
% Omega : frequency in continuous-time domain

Td = 2*tan(omega/2)/Omega;
% uniformly distributed angles
angle = pi/2 + (0.5 : 1 : N-0.5).*pi./N;
s_p = Omega*exp(1j*angle); % look up end of text book
% map s to z
z_p = (1+Td*s_p/2)./(1-Td*s_p/2); % Bilinear Transform
DEN = poly(z_p); % Denominator A(z)

z_zero = -1*ones(1,N);
NUM = poly(z_zero); % Numerator B(z)

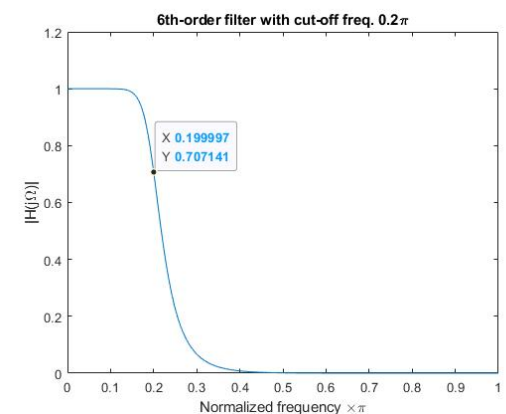
DCgain = 2^N/sum(DEN); % B(z)/A(z) evaluated at z = 1, i.e. @ omega = 0.
NUM = NUM/ DCgain;
end
```

2.

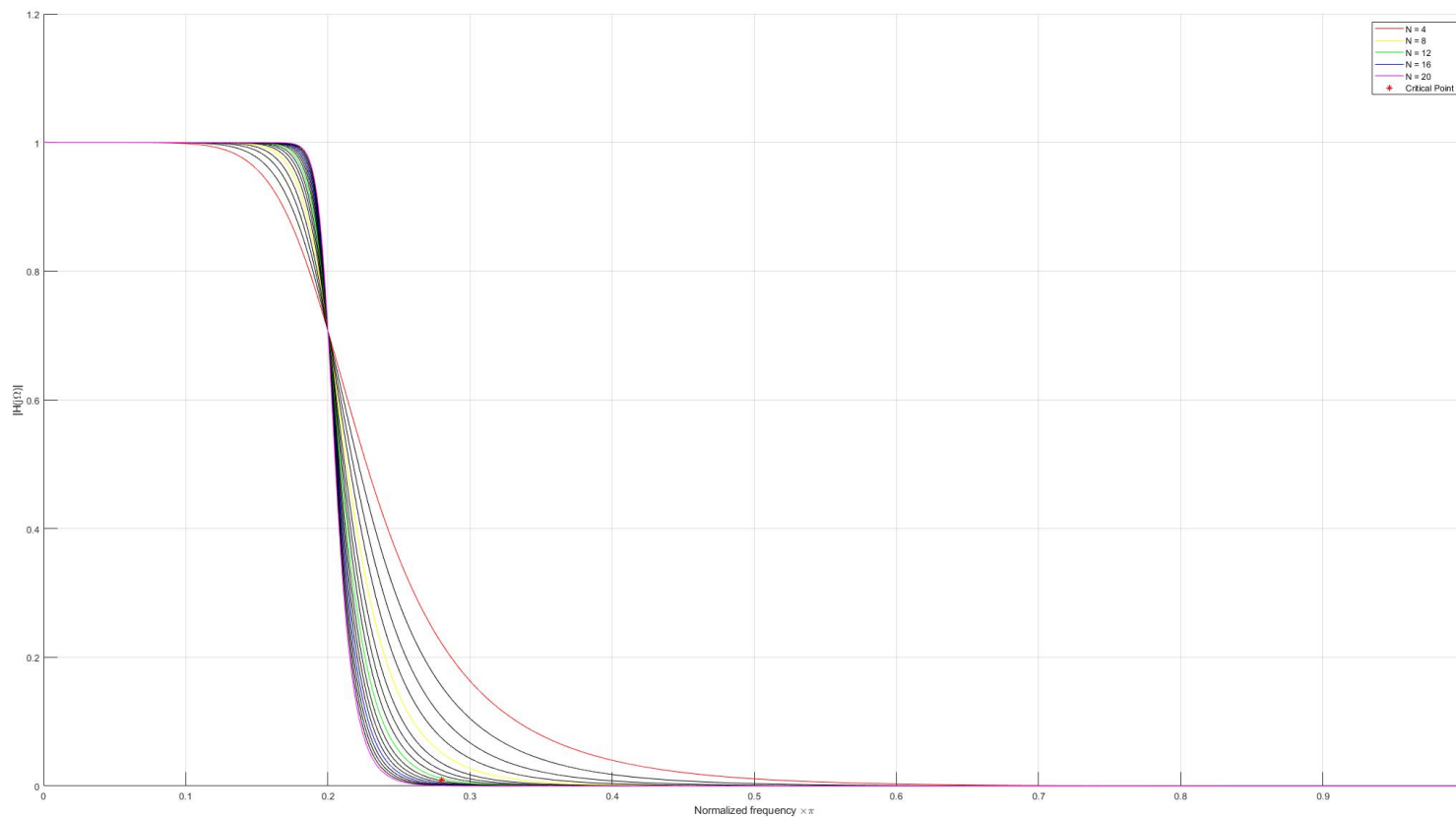
A Butterworth low-pass filter has the form

$$|H_c(j\Omega)|^2 = \frac{1}{1 + (j\Omega/j\Omega_c)^{2N}} \quad (2)$$

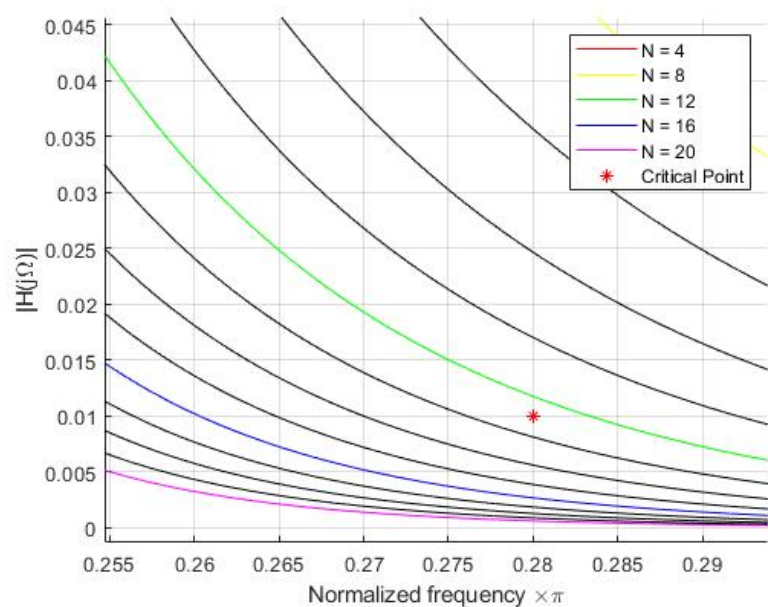
We know that the magnitude response of a Butterworth low-pass filter monotonically falls off in the passband and stopband, so it would always be equal to $\frac{1}{\sqrt{2}}$ or 0.7071 at the cut-off frequency because of the nature of the above form. The magnitude response of a 6th-order filter with $\Omega_c = 0.2\pi$ is shown on the right side. It drops to 0.707141 at about 0.2π as inference.



3. Initially, when $N = 4$, the red curve is gentle at the edge. As the parameter N increases from 4 to 20, equivalently the order of color of a rainbow, as the picture shows below. The filter characteristics become sharper. That implies it is more like an ideal LPF as N approaches to infinity. Another interesting is a converging point at about 0.2π , that is exactly the cut-off frequency.



4. Zoom in the region around 0.28π . Obviously, it can achieve our goal when N is greater than 12, so the answer is 13.



```
function [N, DEN, NUM] = FunctionForQ5(lb, ub, pb, sb, Td)
% passband ≤ |H(e^jw)| ≤ 1,      0 ≤ |w| ≤ lower bound
% |H(e^jw)| < stopband, upbound ≤ |w| ≤ π
% lb : lower bound
% ub : upper bound
% pb : passband
% sb : stopband

N = ceil(log(((1/sb)^2-1)/((1/pb)^2-1))/(2*log(tan(ub/2)/tan(lb/2))));
Omega_c = 2*tan(lb/2)/Td/(1/pb^2-1)^(1/2/N);
[~, DEN, NUM] = FunctionForQ1(N, lb, Omega_c);
end
```

5.

Define

lower bound of frequency (cutoff/pass frequency) : lb

upper bound of frequency (stop frequency) : ub

passband magnitude response : pb

stopband magnitude response : sb

Bilinear transformation maps the discrete-time frequencies back to the corresponding continuous-time critical frequencies. That is

$$pb \leq |H_c(j\Omega)| \leq 1, \quad 0 \leq \Omega \leq \frac{2}{T_d} \tan(lb/2)$$

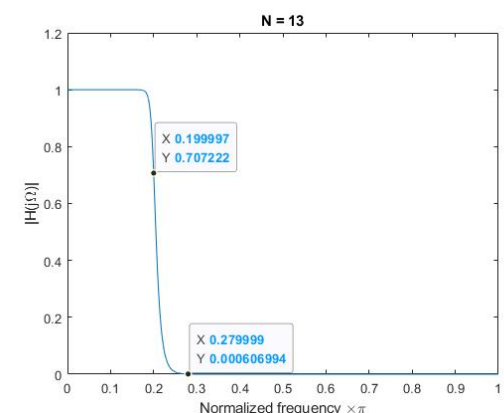
$$|H_c(j\Omega)| \leq sb, \quad \frac{2}{T_d} \tan(ub/2) \leq \Omega \leq \infty$$

Since a Butterworth filter has a monotonic magnitude response, we can equivalently require that

$$|H_c(j2\tan(lb))/T_d| \geq pb$$

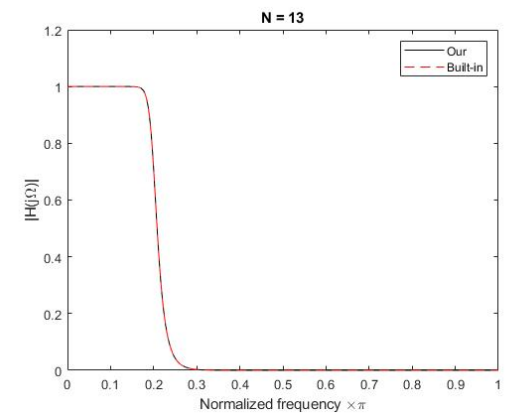
$$|H_c(j2\tan(ub))/T_d| \leq sb$$

Considering the Butterworth filter form equation (2) mentioned in question 2, we obtain



$$1 + \left(\frac{2\tan(lb/2)}{T_d\Omega_c}\right)^{2N} = \left(\frac{1}{pb}\right)^2 \quad (3)$$

$$1 + \left(\frac{2\tan(ub/2)}{T_d\Omega_c}\right)^{2N} = \left(\frac{1}{sb}\right)^2 \quad (4)$$



Solve for N. Note that N is required to be an integer.

$$N = \text{ceil}\left(\frac{\log\left[\left(\left(\frac{1}{pb}\right)^2 - 1\right) / \left(\left(\frac{1}{sb}\right)^2 - 1\right)\right]}{2\log\left[\tan(lb/2) / \tan(ub/2)\right]}\right)$$

Follow the above steps, FunctionForQ5 automatically computes N and then updates the numerator and denominator in reference to new Ω_c using FunctionForQ1. Since we round N to an integer, at the expense of the accuracy of ω by equation (1), either cut-off frequency or stop frequency is accurate depending solving from which equation. A precise cut-off frequency is imperative, so we further intend to derive Ω_c using equation (3). Namely, magnitude response at stop frequency might inaccurate, instead of 0.01 here, but cut-off frequency is exactly equal to our desired value. In details, we get magnitude response 0.00811065 at 0.28π in question 3 while it becomes 0.000606994 here. Assume that cut-off frequency is 0.2π , stop frequency is 0.28π , and the associated magnitude response is 0.7071 and 0.001, so N derived from FunForQ5 is **13**. The resultant N is the same as that observed directly in question 3, and its corresponding magnitude response is shown on the top right which suffices our design. Then we compare our design with filter that from built-in function as the bottom right figure shows. They are almost the same.

6.

The form of $H(z)$ can be represented as

$$H(z) = \frac{Y(z)}{X(z)} = C \prod_{k=1}^N \frac{(1 + q_k z)}{(1 - p_k z)},$$

where C is a constant.

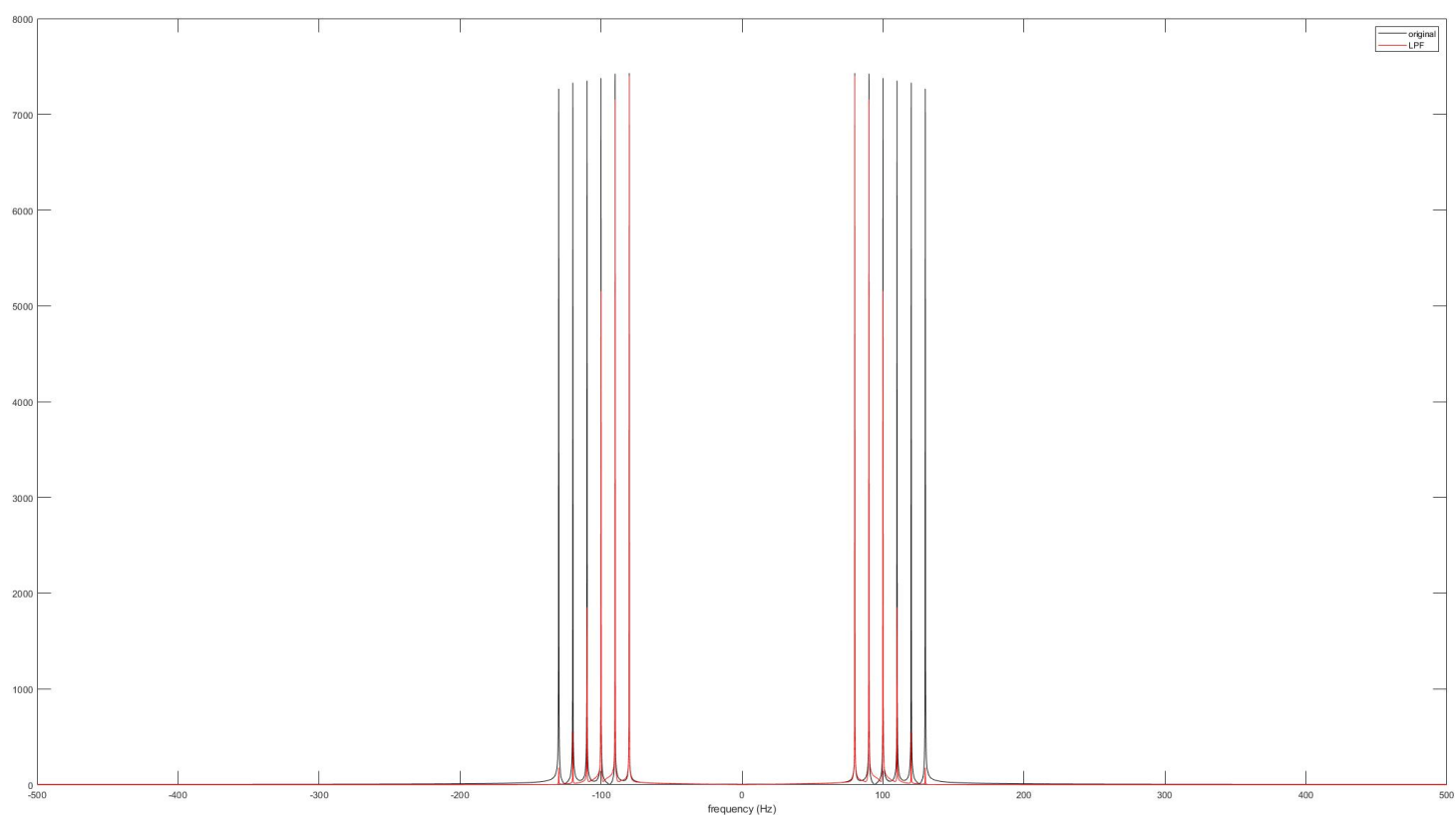
Reformulate z terms into

```
function y = FunctionForQ6(NUM,DEN,x)
    ytermLen = length(DEN(2:end));
    xtermLen = length(NUM);
    x = reshape(x,[1,length(x)]);
    y = zeros(size(x));
    for ii = 1:length(x)
        if ii == 1
            y(ii) = NUM(1)*x(1)/DEN(1);
        elseif 1<ii && ii<xtermLen
            y(ii) = (sum(NUM(1:ii).*fliplr(x(1:ii)))-sum(DEN(2:ii).*fliplr(y(1:ii-1))))/DEN(1);
        else
            y(ii) = (sum(NUM.*fliplr(x(ii-xtermLen+1:ii)))-sum(DEN(2:end).*fliplr(y(ii-ytermLen:ii-1))))/DEN(1);
        end
    end
end
```

$$z^N Y(z) = (c'_n z^N + c'_{n-1} z^{N-1} + \dots + c')X(z) + (c_{n-1} z^{N-1} + c_{n-2} z^{N-2} + \dots + c)Y(z)$$

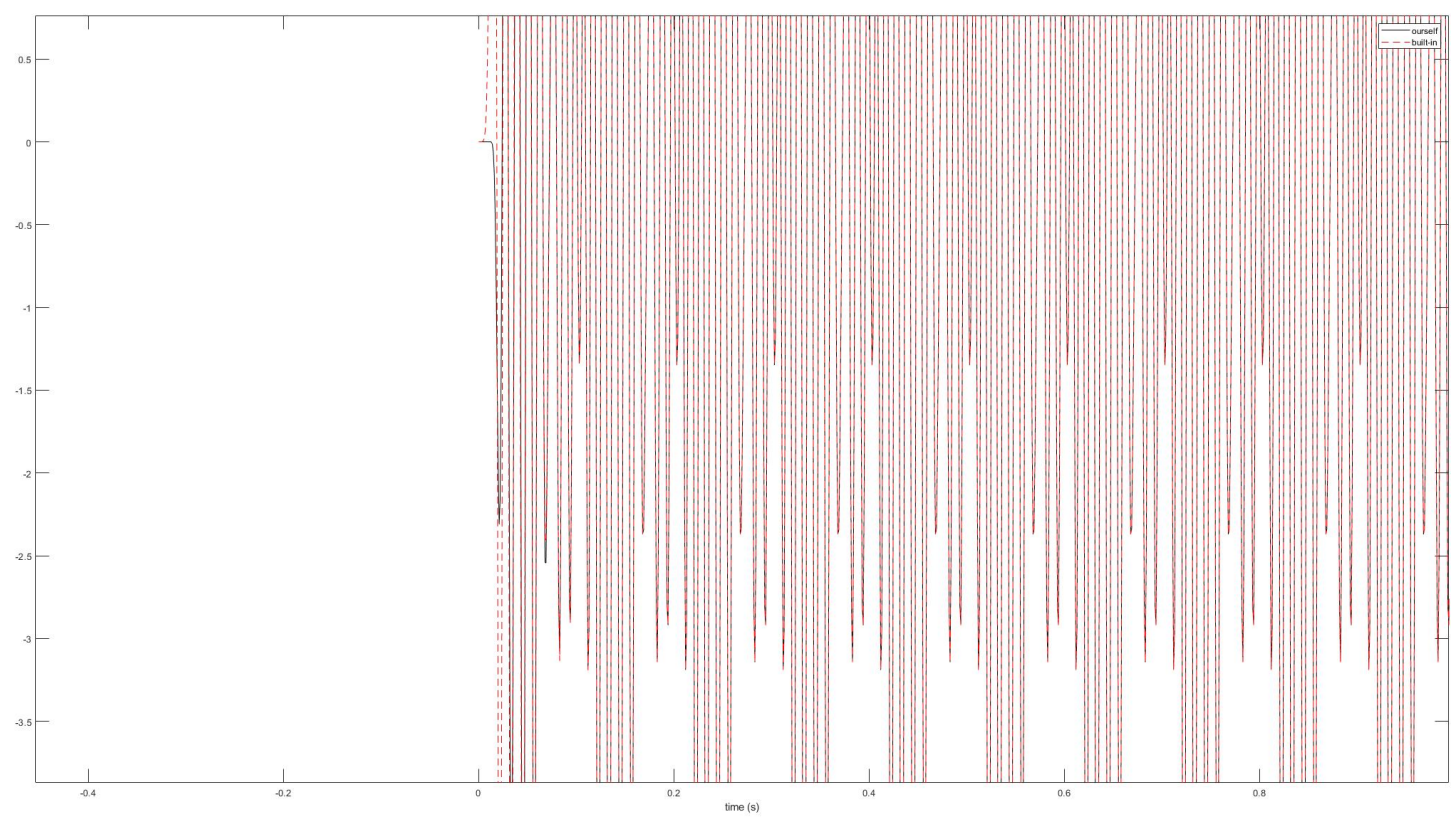
$$y[n] = c'_n x[n] + c'_{n-1} x[n-1] + \dots + c' x[n-N+1] + c_{n-1} y[n-1] + c_{n-2} y[n-2] + \dots + c y[n-N]$$

To verify its functionality, we send a test signal to FunctionForQ6 which consists of different frequencies from 80 Hz to 130Hz. The spectrum is shown below. Sampling rate is set as 1000 Hz, so the cut-off frequency is equivalently 100 Hz. A significant fall at 100 Hz indicates our difference equation works perfectly.

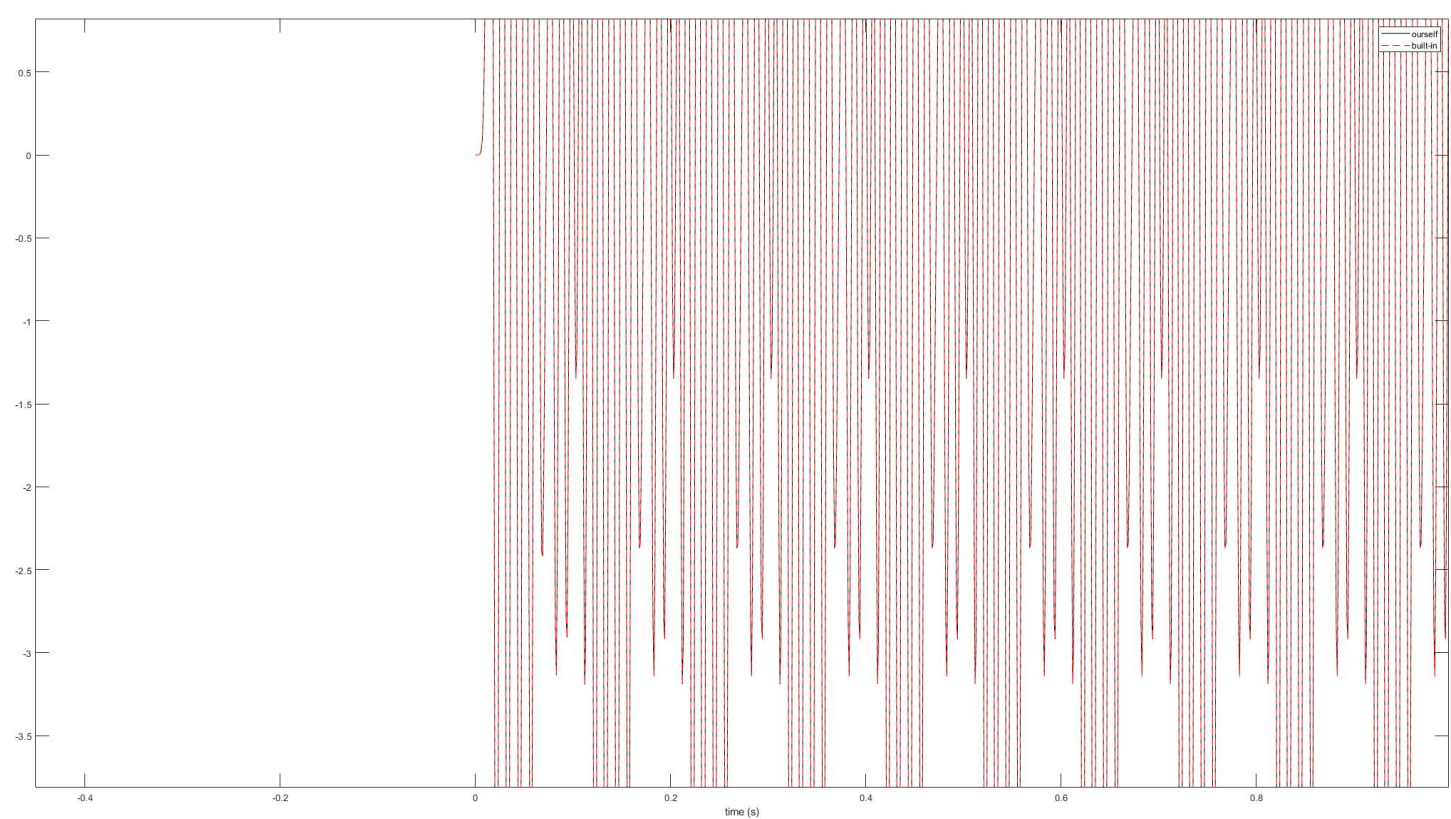


Thanks for TA's reminding, initial values of the first N outputs cannot be directly set as zero that would lead to a little inconsistent outputs. I try to reset the values of x and y as zero at negative n instead of the first N terms and derive outputs again. Now the overcome is similar to that obtained from the built-in

function *filter*.



If the first N outputs are set as zero.



values at negative n are zero only

A file named *boxing.wav* in HW4 is reused to test here. Before processing, it has been a clip with appropriate length. Now we only need to send this clip to FunctionForQ6. All parameters of Butterworth filter are referred to question 5. To see the differences between the original signal and signal applied LPF, their spectra are drawn in the same figure on the right side. Let's convert normalized frequency to Hz, that the cut-off frequency 0.2π is equal to $0.2 \times f_s/2$, where f_s is the sampling frequency, e.g. 44100Hz. So it should have an obvious drop at about 4410 Hz theoretically. As the right figure shows, frequencies over about 5 kHz are suppressed fiercely. We further zoom in the region around 4410 Hz to justify our inference, that is shown below. The steep slope does credit to the high-order N such that frequencies over 4410 Hz can be reduced quickly.

